IE540
L6. n-Step TD Methods

KAIST ISE
Hayong Shin

Overview

- Model is known → DP (in Lecture 3)
  - Evaluation (prediction) : Iterative policy evaluation
  - Control (finding optimal solution)
    - PI → GPI → VI
    - Async. DP
- Model is unknown → RL (by use of interaction samples)
  - Value based RL
    - MC method : Evaluation, Control (in Lecture 4)
    - TD method : Q, SARSA (in Lecture 5)
    - n-step bootstrapping (in Lecture 6)
    - value function approximation, DQN (in Lecture 7)
  - Policy based RL (Lecture 8~10)
    - Derivative free methods
    - Policy gradient

Review

MC for continuing task ($\gamma < 1$) : delayed update

$\gamma^m \approx 0$

$\tau =$

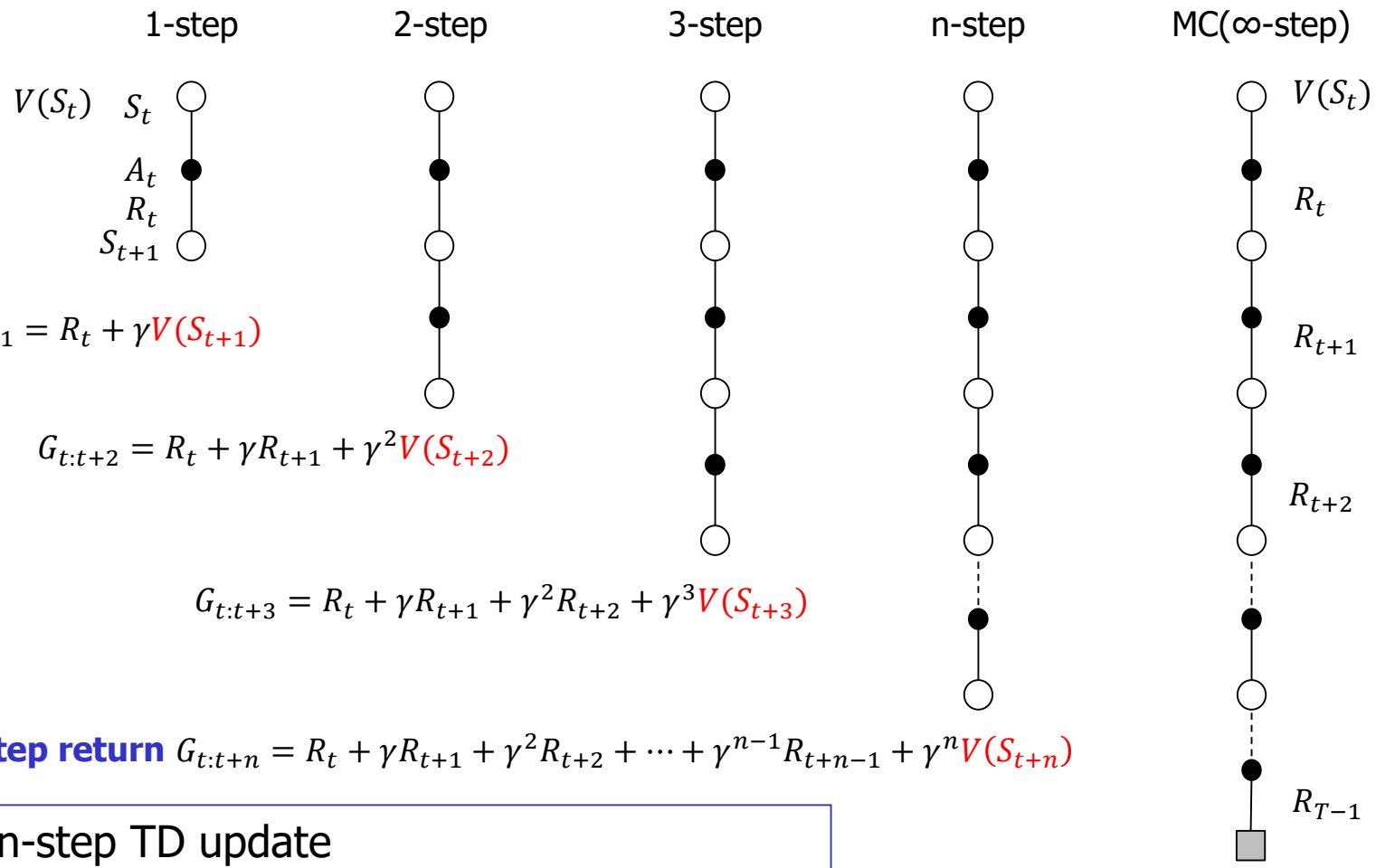
S_0	S_1	S_2	S_3	..	S_m	S_{m+1}
a_0	a_1	a_2	a_3	..	a_m	a_{m+1}
r_0	r_1	r_2	r_3		r_m	r_{m+1}

- Policy Evaluation (Prediction)
  - DP backup
    - $V(s) \leftarrow \mathbb{E}_{\substack{a\sim\pi \\ s'\sim p(.|s,a)}}[R(s,a)+\gamma V(s')] = \sum_{a\in\mathcal{A}} \pi(a|s)\left[\ R(s,a)+\gamma \sum_{s'} p(s'|s,a)V(s')\ \right]$
  - MC backup : $V(s) \overset{\alpha}{\leftarrow} G(s)$
  - TD(0) backup : $V(s) \overset{\alpha}{\leftarrow} r+\gamma V(s')$
- Optimal value (Control)
  - DP (Value iteration)
    - $Q(s,a) \leftarrow R(s,a)+\gamma \mathbb{E}_{s'\sim p(.|s,a)}\left[\max_{a'} Q(s',a')\right]$
  - MC control : $Q(s,a) \overset{\alpha}{\leftarrow} G(s,a)$
  - TD(0) control
    - SARSA : $Q(s,a) \overset{\alpha}{\leftarrow} r+\gamma Q(s',a')$, $a' \sim$
    - Expected SARSA : $Q(s,a) \overset{\alpha}{\leftarrow} r+\gamma \mathbb{E}_{a'\sim\pi}[Q(s',a')]$, $\pi = \epsilon-\text{greedy}(Q)$
    - Q-learning : $Q(s,a) \overset{\alpha}{\leftarrow} r+\gamma \max_{a'} Q(s',a') = r+\gamma Q\left(s', \arg\max_{a'} Q(s',a')\right)$
    - Double Q-learning : $Q^A(s,a) \overset{\alpha}{\leftarrow} r+\gamma$

$G(s)$: sample return from state s

$G(s,a)$: sample return from (s,a)

n-step return



$G_{t:t+1} = R_t + \gamma V(S_{t+1})$

$G_{t:t+2} = R_t + \gamma R_{t+1} + \gamma^2 V(S_{t+2})$

$G_{t:t+3} = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 V(S_{t+3})$

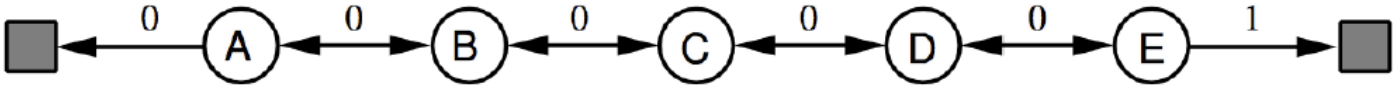
n-step return $G_{t:t+n} = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^{n-1} R_{t+n-1} + \gamma^n V(S_{t+n})$

- n-step TD update
  - $V(S_t) \overset{\alpha_t}{\leftarrow} G_{t:t+n} \equiv V(S_t) + \alpha_t\big(G_{t:t+n} - V(S_t)\big)$

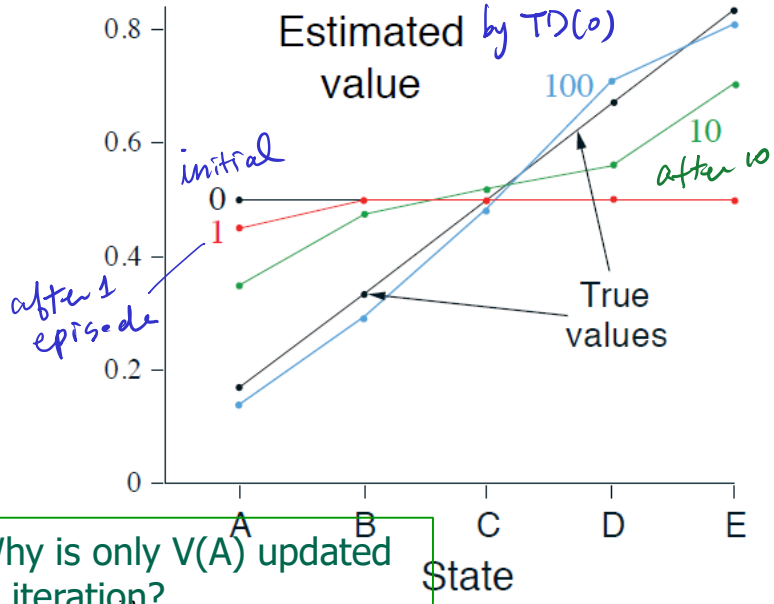
$V(S_T) = 0$

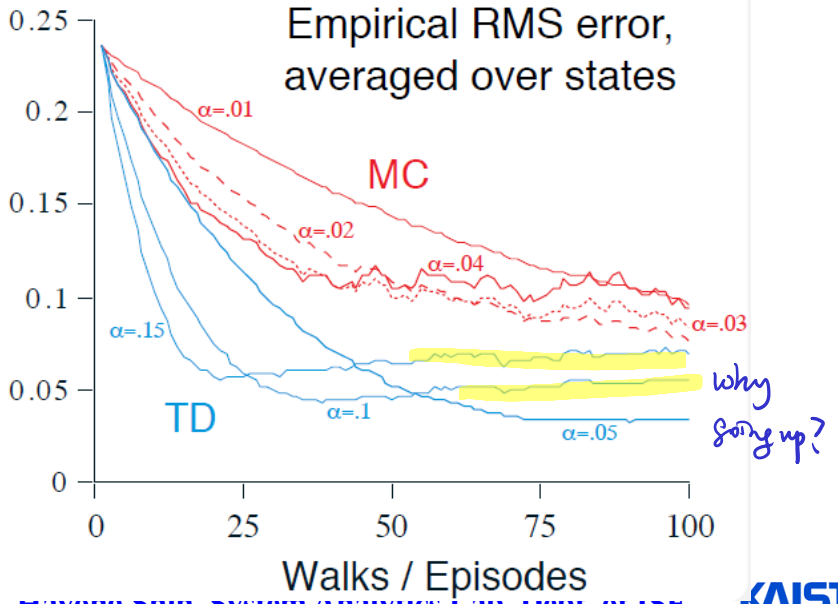
$G_t = R_t + \gamma R_{t+1} + \cdots + \gamma^{T-t-1} R_{T-1}$
$G_{t:k} = G_t \text{ if } k \geq T$

Revisit example : Random walk MRP (RLAI ex.6.2)



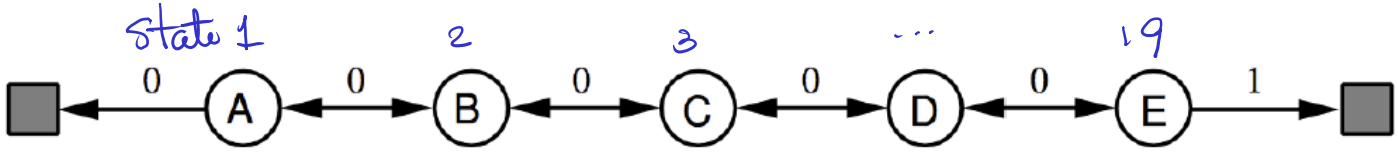
- State S = {A,B,C,D,E,■}, Transition : left or right equally
- Reward : 1 for moving from E to ■, 0 otherwise, $\gamma = 1$
- True value $\left(V(A), V(B), V(C), V(D), V(E)\right) = \left(\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}, \frac{5}{6}\right)$
- Comparing MC vs TD(0)
  - initial $V(*) = 0.5$, learning rate α : constant
  - TD(0) converges faster
  - constant α hinders convergence below some limit





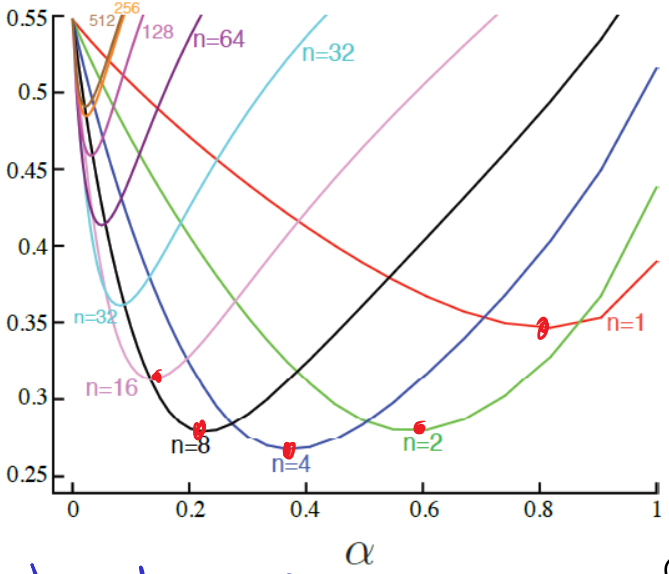
TAQ Why is only V(A) updated after 1 iteration?

n-step TD for Random walk example (19 states) : RLAI Ex.7.1



$$RMSE = \sqrt{\frac{1}{19}\sum_{s=1}^{19}\left(V(s) - V^{\pi}(s)\right)^2}$$

Average RMS error over 19 states and first 10 episodes



Performance of n-step TD update (after 10 episodes)

$n = 1, 2, 4, 8, \cdots, 256, 512$

$\alpha = 0 \sim 1$

→ initial $V(s) = 0.5$ (meaningless)

→ during the first episode the only meaningful information comes when it reaches ⚠

- → $n=1 \Rightarrow V(18)$ is updated
- → $n=2 \Rightarrow V(17)$ is updated

⇒ best learning α^* depends on n

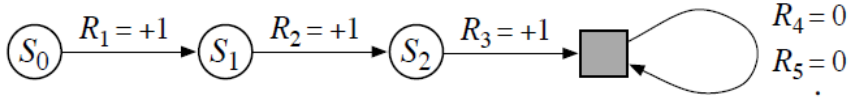
⇒ $n=4$ shows the best performance (initial performance after 10 episodes)

larger n → higher variance, lower bias

smaller n → lower variance, higher bias

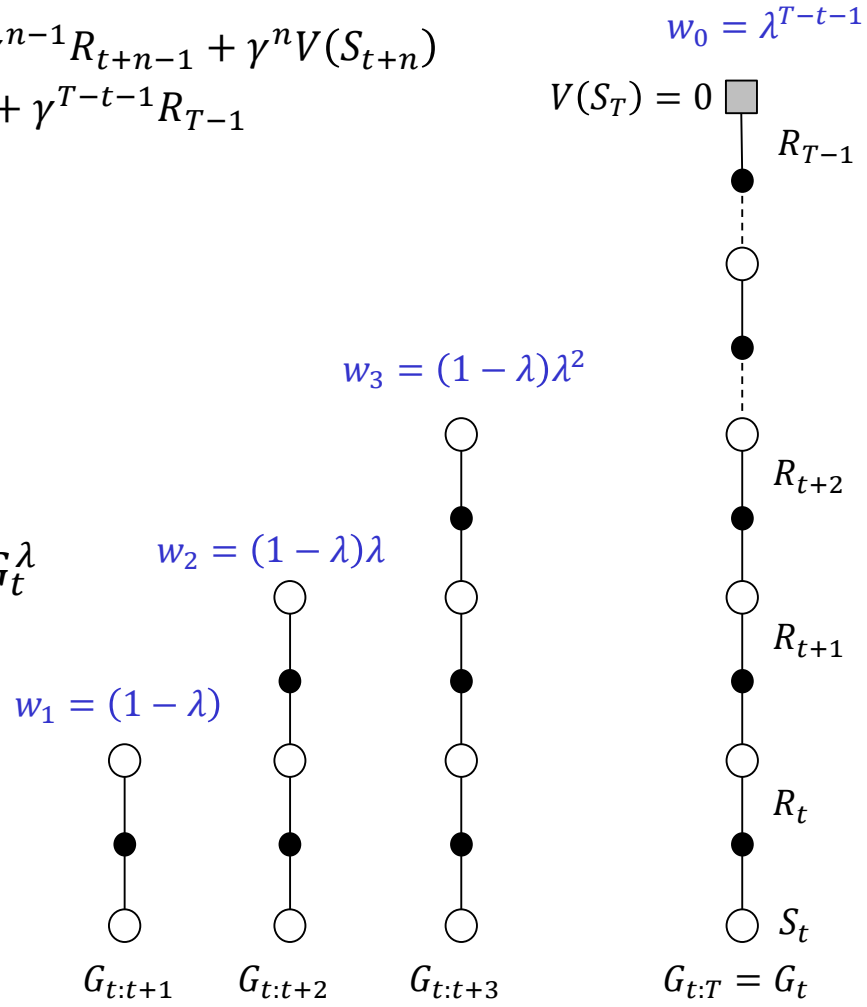
Which n to use?

Unified notation for episodic and continuing task



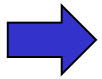
$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

- n-step return
  - $G_{t:t+n} = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^{n-1} R_{t+n-1} + \gamma^n V(S_{t+n})$
  - if $k \geq T$, $G_{t:k} = G_t = R_t + \gamma R_{t+1} + \cdots + \gamma^{T-t-1} R_{T-1}$
- Which n to use?
  - n=∞ : MC
  - n=1 : TD(0)
  - n=?
- Averaging n-step returns : λ-return, G_t^λ
  - $G_t^\lambda = \sum_{n=1}^{\infty} w_n G_{t:t+n}$
  - $w_n = (1-\lambda)\lambda^{n-1}$
    - see next slide



λ-return : exponentially weighted average of n-step returns

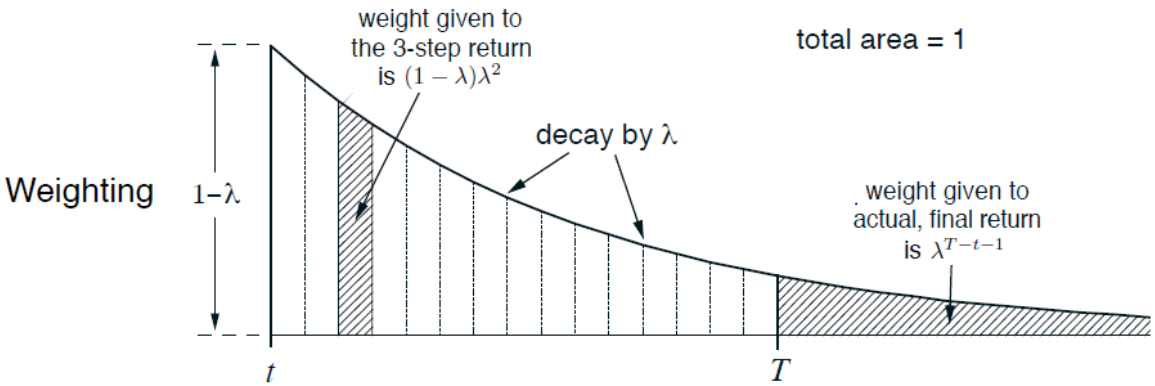
- How to determine w_n
  - $w_n = A\lambda^{n-1}$
    - geometric weighting
    - = exponential weighting
  - $\sum_{n=1}^{\infty} w_n = 1$: weights sum to 1



$w_n =$ [ ]

- Continuing task case : $G_t^{\lambda} = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$
- Episodic task case
  - w_0 : weight for $G_t = G_{t:T} = G_{t:T+1} = \cdots$: final return
    - $w_0 = \sum_{n=T-t}^{\infty} w_n = \frac{(1-\lambda)\lambda^{T-t-1}}{1-\lambda} = \lambda^{T-t-1}$
  - $G_t^{\lambda} = (1-\lambda)\sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_t$

until termination　　after termination

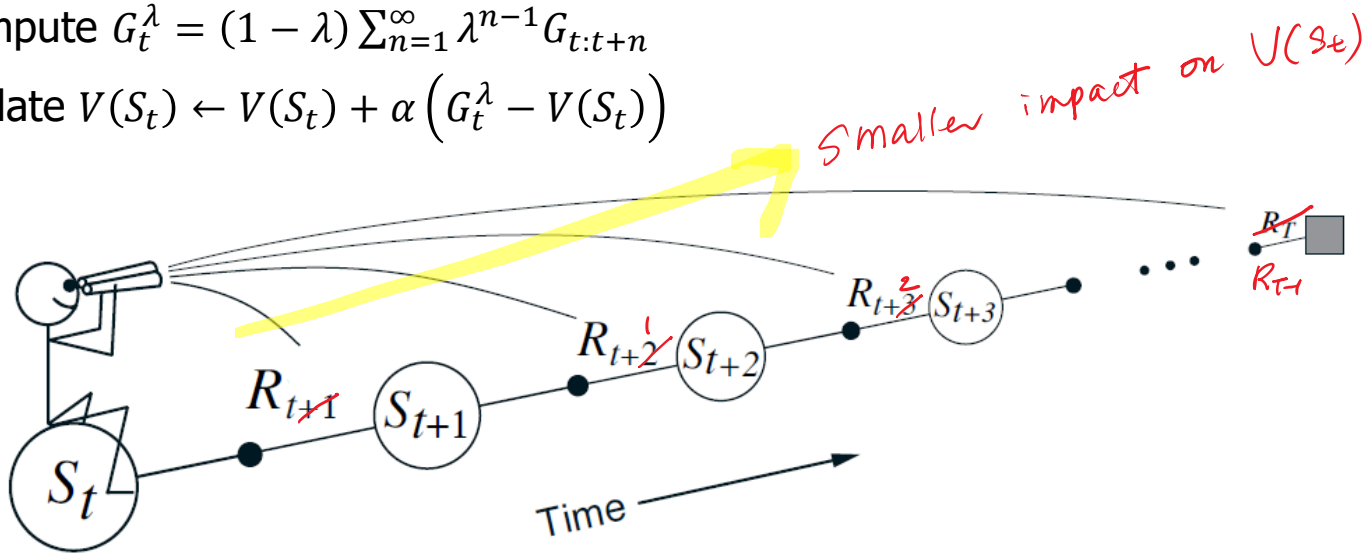


KAIST

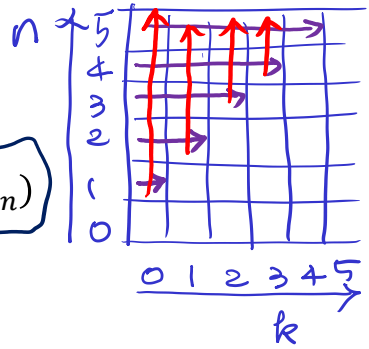
TD(λ)

- TD(λ) vs TD(0) vs TD(1)
  - $G_t^\lambda = (1-\lambda)\sum_{n=1}^{T-t-1}\lambda^{n-1}G_{t:t+n} + \lambda^{T-t-1}G_t$
  - $\lambda = 0 : w_1 = 1, w_2 = w_3 = \cdots = 0$
    → TD(0) = 1-step TD
  - $\lambda = 1 : w_0 = 1, w_1 = w_2 = \cdots = 0$
    → TD(1) = MC

- TD(λ) Policy Evaluation : Forward view of TD(λ)
  - generate an episode: $\tau = (S_0, A_0, R_0, S_1, A_1, R_1, \ldots, S_T) \sim \pi$
  - for each time t,
    - compute $G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_{t:t+n}$
    - update $V(S_t) \leftarrow V(S_t) + \alpha\left(G_t^\lambda - V(S_t)\right)$



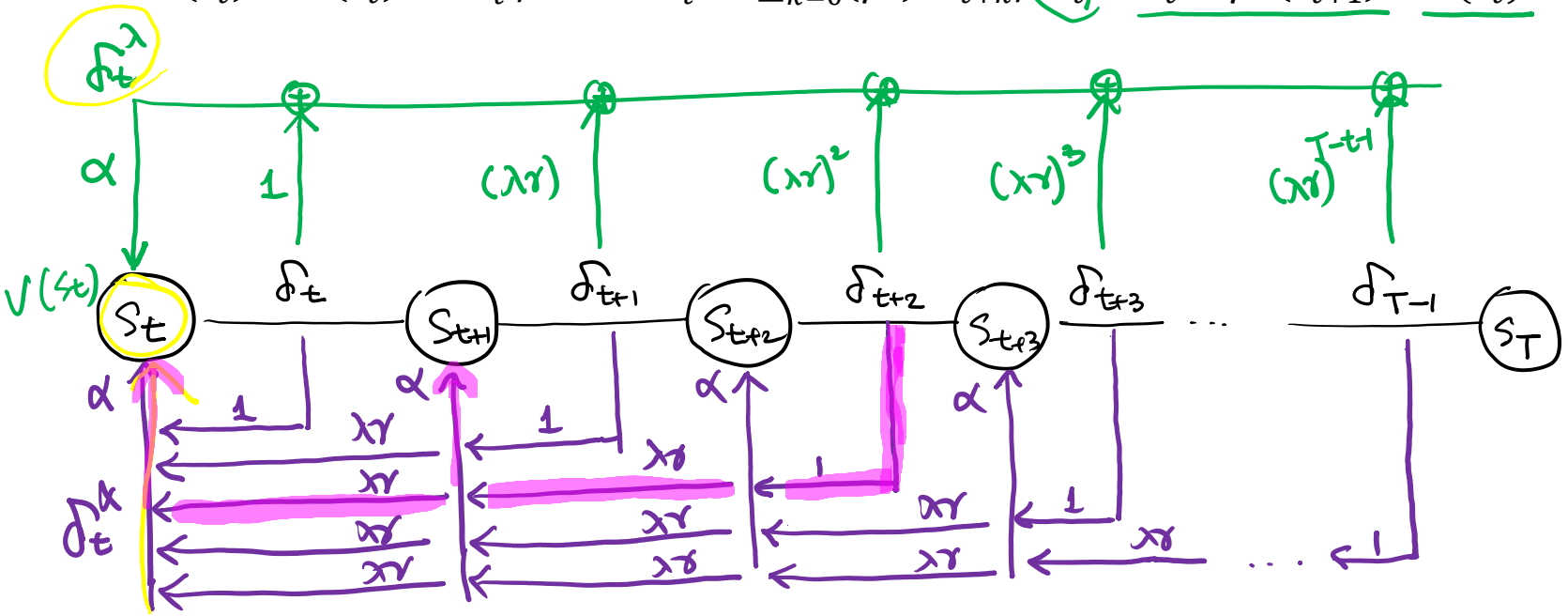
Anatomy of TD(λ) : forward view



- n-step return, λ-return
  - $G_{t:t+n} = R_t + \gamma R_{t+1} + \cdots + \gamma^{n-1} R_{t+n-1} + \gamma^n V(S_{t+n}) = \sum_{k=0}^{n-1} \gamma^k R_{t+k} + \gamma^n V(S_{t+n})$
  - $G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$
- TD(λ) update : $V(S_t) \leftarrow V(S_t) + \alpha \delta_t^\lambda$, where $\delta_t^\lambda = G_t^\lambda - V(S_t)$
- $\delta_t^\lambda = G_t^\lambda - V(S_t) = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$, where $\delta_t = R_t + \gamma V(S_{t+1}) - V(S_t)$: 1-step TD error

Switching from forward view to backward view

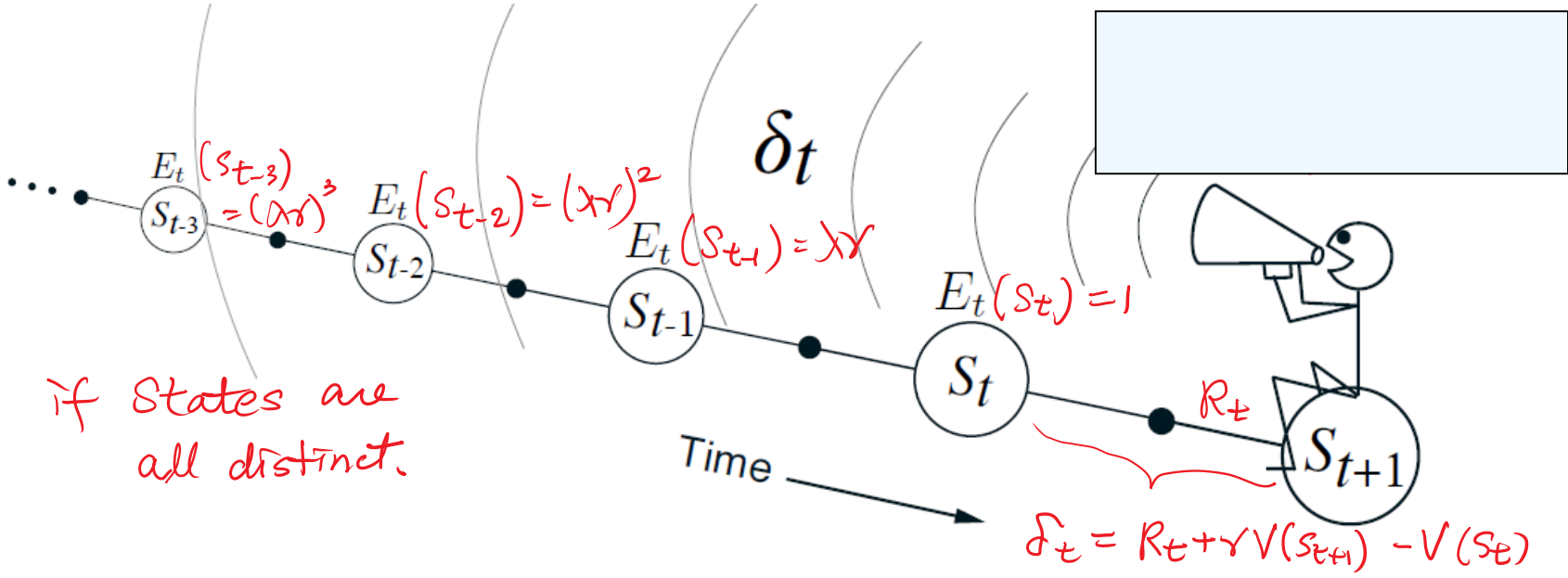
- Forward view (for theory)
  - $V(S_t) \leftarrow V(S_t) + \alpha\delta_t^\lambda$, where $\delta_t^\lambda = \sum_{k=0}^{\infty}(\gamma\lambda)^k\delta_{t+k}$, $\delta_t = R_t + \gamma V(S_{t+1}) - V(S_t)$



- Backward view (implementation mechanism)
  - update at every step
  - incomplete episode → applicable to continuing task
  - backward view = forward view for offline update case
    - backward view ≈ forward view for online update case
      - online update performs better

Backward view of TD(λ)

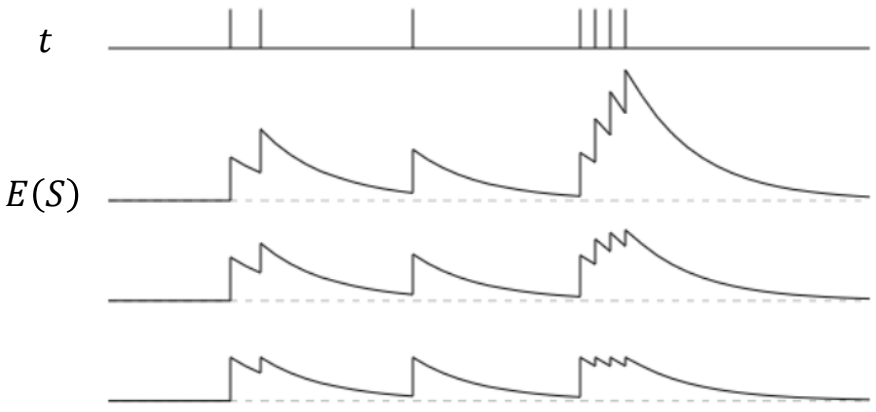
- at each step, $\delta_t = R_t + \gamma V(S_{t+1}) - V(S_t)$ is computed
  - store state trajectory $\tau = (S_0, S_1, S_2, \dots, S_t, S_{t+1})$
    - $V(S_{t-k}) \leftarrow V(S_{t-k}) + \alpha(\lambda\gamma)^k \delta_t$
    - better if state space is large and/or episode is short
  - eligibility trace : leave footprint on the trajectory
    - $E(s) = (\lambda\gamma)^k$ if k time step elapsed since the last visit to s
      - amount of eligibility/responsibility to the TD error δ_t
      - credit assignment
    - $V(s) \leftarrow V(s) + \alpha E(s)\delta_t$ for $\forall s$ with $E(s) > 0$



TD(λ) Algorithm for PE (Backward view, online update)

➢ **Algorithm TD(λ) Policy Evaluation**

- initialize $V(s)$ to any value for $\forall s$ and $V(terminal) \leftarrow 0$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize $E(s) \leftarrow 0$ for $\forall s$
  - initialize S
  - repeat for each step (until S is $terminal$)
    - choose action $A \sim \pi(.|S)$
    - take action A, observe R, S'
    - $\delta =$ [ ]
    - set $E(S) \leftarrow E(S) + 1$ (accumulating trace)
    - for $\forall s$ with $E(s) > 0$
      - $V(s) \leftarrow V(s) + \alpha E(s)\delta$
      - $E(s) \leftarrow$ [ ]



times of visiting state S

accumulating trace $E(S) \leftarrow E(S) + 1$

$\approx$ [ ] (in analogy)

dutch trace $E(S) \leftarrow (1 - \alpha)E(S) + 1$

replacing trace $E(S) \leftarrow 1 \approx$ [ ] (in analogy)

Decay $E(s) \leftarrow \lambda\gamma E(s)$ at each time step

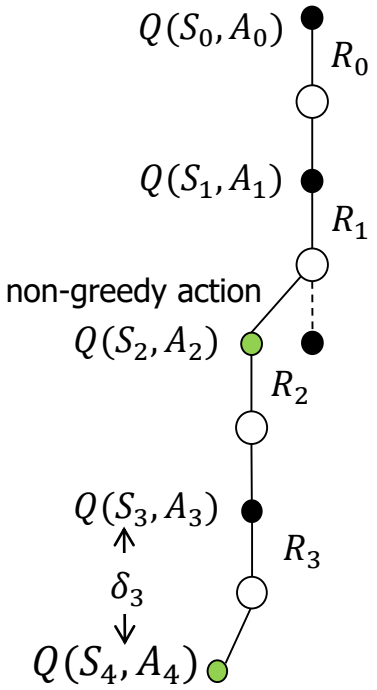
Control : SARSA(λ)

➢ **Algorithm SARSA(λ) On-policy Control**

- initialize $Q(s,a)$ to any value for $\forall(s,a)$ and $Q(terminal,*) \leftarrow 0$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize $E(s,a) \leftarrow 0$ for $\forall s,a$
  - initialize S, choose $A \sim \epsilon$–greedy(Q) at S
  - repeat for each step (until S is $terminal$)
    - take action A, observe R, S'
    - choose next action $A' \sim \epsilon$–greedy(Q) at S'
    - $\delta \leftarrow$
    - set $E(S,A) \leftarrow E(S,A) + 1$ (accumulating trace)
    - for $\forall s,a$ with $E(s,a) > 0$
      - $Q(s,a) \leftarrow Q(s,a) + \alpha E(s,a)\delta$
      - $E(s,a) \leftarrow \lambda\gamma E(s,a)$
    - $S \leftarrow S',\ A \leftarrow A'$

SARSA(λ)

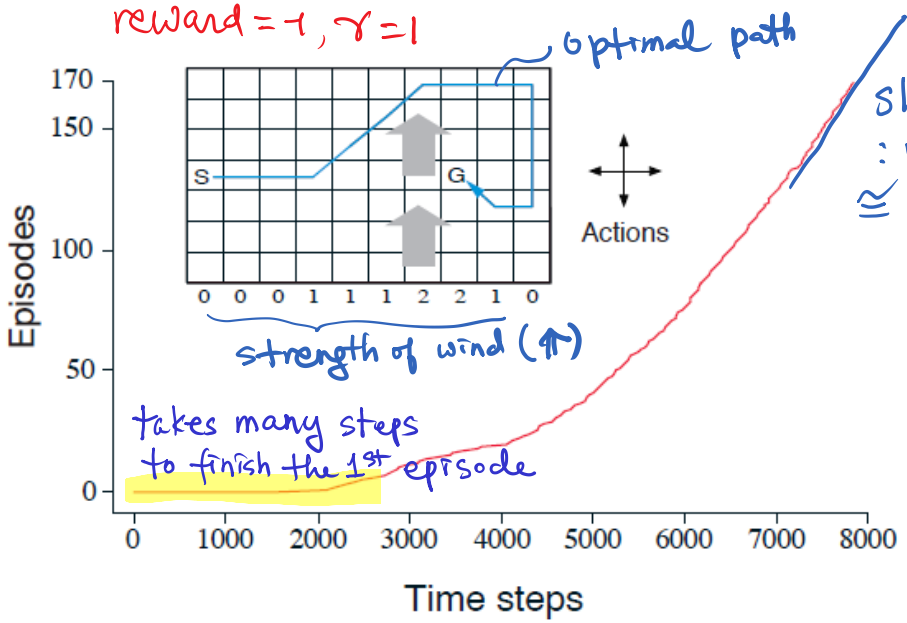
- Scenario $(\gamma = 1, \lambda = 0.9)$
  - $t = 0 : (S_0, A_0, R_0, S_1, A_1) \rightarrow \delta_0 = R_0 + \gamma Q(S_1, A_1) - Q(S_0, A_0)$
    - $E(S_0, A_0) = 1$
    - $Q(S_0, A_0)$ += $\alpha \delta_0$, $E(S_0, A_0) = 0.9$
  - $t = 1 : (S_1, A_1, R_1, S_2, A_2) \rightarrow \delta_1 = R_1 + \gamma Q(S_2, A_2) - Q(S_1, A_1)$
    - $E(S_1, A_1) = 1$
    - $Q(S_0, A_0)$ += $\alpha(0.9)\delta_1$, $E(S_0, A_0) = (0.9)^2$
    - $Q(S_1, A_1)$ += $\alpha(1)\delta_1$, $E(S_1, A_1) = 0.9$
  - $t = 2 : (S_2, A_2, R_2, S_0, A_0) \rightarrow \delta_2 = R_2 + \gamma Q(S_0, A_0) - Q(S_2, A_2)$
    - $E(S_2, A_2) = 1$
    - $Q(S_0, A_0)$ += $\alpha(0.9)^2\delta_2$, $E(S_0, A_0) = (0.9)^3$
    - $Q(S_1, A_1)$ += $\alpha(0.9)\delta_2$, $E(S_1, A_1) = (0.9)^2$
    - $Q(S_2, A_2)$ += $\alpha(1)\delta_2$, $E(S_2, A_2) = (0.9)$
  - $t = 3 : (S_0, A_0, R_3, S_4, A_4) \rightarrow \delta_3 = R_3 + \gamma Q(S_4, A_4) - Q(S_0, A_0)$
    - $E(S_0, A_0) =$
    - $Q(S_0, A_0)$ += $\alpha(1.729)\delta_3$, $E(S_0, A_0) = 1.729 * 0.9 = 1.5561$
    - $Q(S_1, A_1)$ += $\alpha(0.9)^2\delta_3$, $E(S_1, A_1) = (0.9)^3$
    - $Q(S_2, A_2)$ += $\alpha(0.9)\delta_3$, $E(S_2, A_2) = (0.9)^2$



$S_3, A_3 \equiv S_0, A_0$

Windy gridworld (RLAI Ex.6.5)

Example 6.5: Windy Gridworld Shown inset below is a standard gridworld, with start and goal states, but with one difference: there is a crosswind running upward through the middle of the grid. The actions are the standard four—`up`, `down`, `right`, and `left`—but in the middle region the resultant next states are shifted upward by a "wind," the strength of which varies from column to column. The strength of the wind is given below each column, in number of cells shifted upward. For example, if you are one cell to the right of the goal, then the action `left` takes you to the cell just above the goal. This is an undiscounted episodic task, with constant rewards of -1 until the goal state is reached.



The graph to the right shows the results of applying ε-greedy Sarsa to this task, with $\varepsilon = 0.1$, $\alpha = 0.5$, and the initial values $Q(s,a) = 0$ for all s, a. The increasing slope of the graph shows that the goal was reached more quickly over time. By 8000 time steps, the greedy policy was long since optimal (a trajectory from it is shown inset); continued ε-greedy exploration kept the average episode length at about 17 steps, two more than the minimum of 15. Note that Monte Carlo methods cannot easily be used here because termination is not guaranteed for all policies. If a policy was ever found that caused the agent to stay in the same state, then the next episode would never end. Online learning methods such as Sarsa do not have this problem because they quickly learn *during the episode* that such policies are poor, and switch to something else. ■

taking 'down' action when wind = +1 → agent stays.

No-wind gridworld

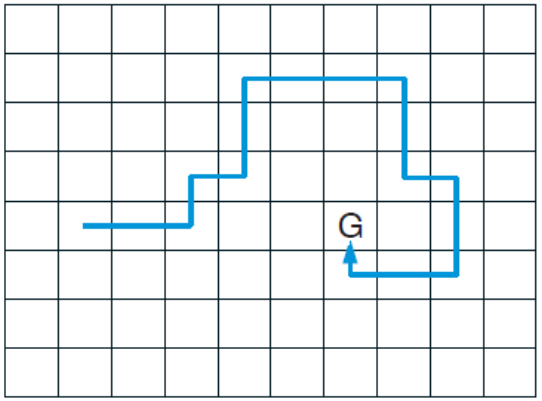
reward = +100 if $s' = G$, 0 otherwise

$\gamma = 0.9$

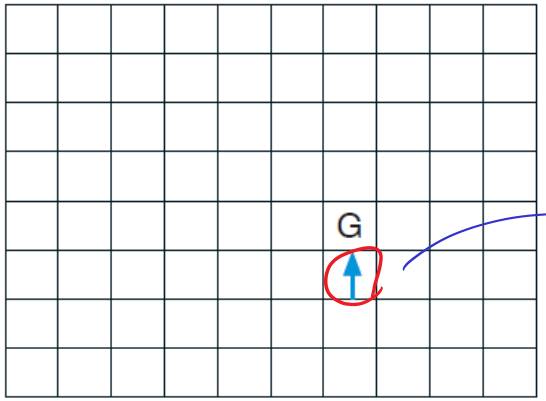
This example assumes No Wind.

(1st successful path)

Path taken



Action values increased by one-step Sarsa



initial $Q(s,a) = 0$

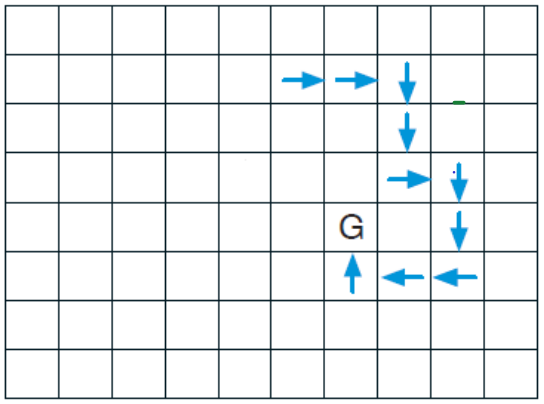
$\delta = R + \gamma Q(S',A') - Q(S,A)$

$Q(S,A) += \alpha\delta$

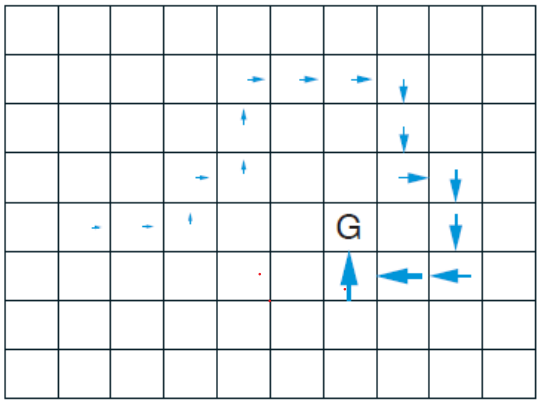
information on G does not flow back along the path taken

$\delta = \begin{cases} +100 & \text{(final step)} \\ 0, & \text{elsewhere} \end{cases}$

Action values increased by 10-step Sarsa



Action values increased by Sarsa(λ) with λ=0.9



TAQ. In 10 step SARSA case, are the arrow size equal for all moves in the path?

info. flows back 10 steps ~~of equal magnitude~~ ($\gamma < 1 \Rightarrow$ amount of increase is bigger near G)

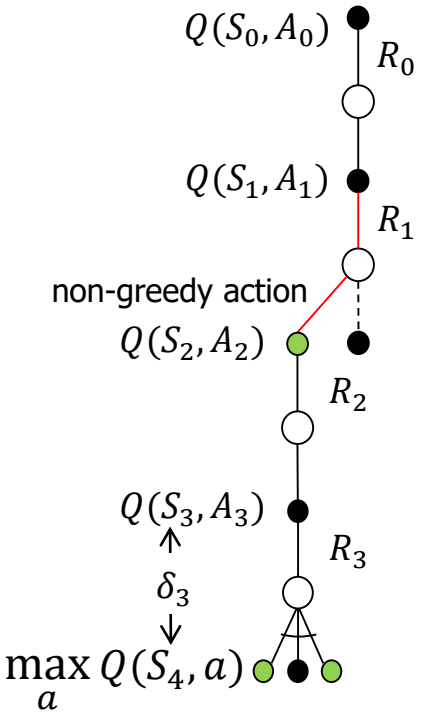
Naïve Q(λ) learning algorithm

➢ **Algorithm Naïve Q(λ) Off-policy Control**

- initialize $Q(s,a)$ to any value for $\forall(s,a)$ and $Q(terminal,*) \leftarrow 0$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize $E(s,a) = 0$ for $\forall s,a$
  - initialize S
  - repeat for each step (until S is $terminal$)
    - choose $A \sim \epsilon$−greedy(Q) at S
    - take action A, observe R, S'
    - $\delta \leftarrow R + \gamma \max_a Q(S',a) - Q(S,A)$
    - set $E(S,A) \leftarrow E(S,A) + 1$ (accumulating trace)
    - for $\forall s,a$ with $E(s,a) > 0$
      - $Q(s,a) \leftarrow Q(s,a) + \alpha E(s,a)\delta$
      - $E(s,a) \leftarrow \lambda\gamma E(s,a)$
    - $S \leftarrow S'$

Naïve Q(λ) learning scenario

- Scenario ($\gamma = 1, \lambda = 0.9$)
  - $t = 0 : (S_0, A_0, R_0, S_1) \rightarrow \delta_0 = R_0 + \gamma \max_a Q(S_1, a) - Q(S_0, A_0)$
    - $E(S_0, A_0) = 1$
    - $Q(S_0, A_0) \mathrel{+}= \alpha\delta_0,\ E(S_0, A_0) = 0$
  - $t = 1 : (S_1, A_1, R_1, S_2) \rightarrow \delta_1 = R_1 + \gamma \max_a Q(S_2, a) - Q(S_1, A_1)$
    - $E(S_1, A_1) = 1$
    - $Q(S_0, A_0) \mathrel{+}= \alpha(0.9)\delta_1\ ,\ E(S_0, A_0) = (0.9)^2$
    - $Q(S_1, A_1) \mathrel{+}= \alpha(1)\delta_1\ ,\ E(S_1, A_1) = 0.9$
  - $t = 2 : (S_2, A_2, R_2, S_3) \rightarrow \delta_2 = R_2 + \gamma \max_a Q(S_3, a) - Q(S_2, A_2)$
    - $E(S_2, A_2) = 1$
    - $Q(S_0, A_0) \mathrel{+}= \alpha(0.9)^2\delta_2\ ,\ E(S_0, A_0) = (0.9)^3$
    - $Q(S_1, A_1) \mathrel{+}= \alpha(0.9)\delta_2\ ,\ E(S_1, A_1) = (0.9)^2$
    - $Q(S_2, A_2) \mathrel{+}= \alpha(1)\delta_2\ ,\ E(S_2, A_2) = (0.9)$
  - $t = 3 : (S_3, A_3, R_3, S_4) \rightarrow \delta_3 = R_3 + \gamma Q(S_4, A_4) - Q(S_3, A_3)$
    - $E(S_3, A_3) = 1$
    - $Q(S_0, A_0) \mathrel{+}= \alpha(0.9)^3\delta_3\ ,\ E(S_0, A_0) = (0.9)^4$
    - $Q(S_1, A_1) \mathrel{+}= \alpha(0.9)^2\delta_2\ ,\ E(S_1, A_1) = (0.9)^3$
    - $Q(S_2, A_2) \mathrel{+}= \alpha(0.9)\delta_3\ ,\ E(S_2, A_2) = (0.9)^2$
    - $Q(S_3, A_3) \mathrel{+}= \alpha(1)\delta_3\ ,\ E(S_3, A_3) = 0.9$



TAQ. Are (S_0, A_0) and (S_1, A_1) eligible (responsible) for TD errors occurring after non-greedy action A_2?

Watkins's Q(λ) learning algorithm

➢ **Algorithm Watkins's Q(λ) Off-policy Control**

- initialize $Q(s,a)$ to any value for $\forall(s,a)$ and $Q(terminal,*) \leftarrow 0$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize $E(s,a) = 0$ for $\forall s,a$
  - initialize S, choose $A \sim \epsilon$–greedy(Q) at S
  - repeat for each step (until S is $terminal$)
    - take action A, observe R, S'
    - choose **next action** $A' \sim \epsilon$–greedy(Q) at S'
    - $\delta \leftarrow R + \gamma \max_a Q(S',a) - Q(S,A)$
    - set $E(S,A) \leftarrow E(S,A) + 1$ (accumulating trace)
    - for $\forall s,a$ with $E(s,a) > 0$
      - $Q(s,a) \leftarrow Q(s,a) + \alpha E(s,a)\delta$
      - if $Q(S',A') = \max_a Q(S',a)$ (i.e. A' is greedy), then $E(s,a) \leftarrow \lambda\gamma E(s,a)$

        else $E(s,a) \leftarrow 0$ (reset all eligibility trace if A' is non-greedy)
    - $S \leftarrow S',\ A \leftarrow A'$

Watkins's Q(λ) learning scenario

- Scenario ($\gamma = 1, \lambda = 0.9$)
  - $t = 0 : (S_0, A_0, R_0, S_1) \rightarrow \delta_0 = R_0 + \gamma \max_a Q(S_1, a) - Q(S_0, A_0)$
    - $E(S_0, A_0) = 1$
    - $Q(S_0, A_0) \mathrel{+}= \alpha\delta_0,\ E(S_0, A_0) = 0$
  - $t = 1 : (S_1, A_1, R_1, S_2) \rightarrow \delta_1 = R_1 + \gamma \max_a Q(S_2, a) - Q(S_1, A_1)$
    - $E(S_1, A_1) = 1,\ A_2 \neq A_2^*$
    - $Q(S_0, A_0) \mathrel{+}= \alpha(0.9)\delta_1\ ,\ E(S_0, A_0) =$
    - $Q(S_1, A_1) \mathrel{+}= \alpha(1.0)\delta_1\ ,\ E(S_1, A_1) =$
  - $t = 2 : (S_2, A_2, R_2, S_3) \rightarrow \delta_2 = R_2 + \gamma \max_a Q(S_3, a) - Q(S_2, A_2)$
    - $E(S_2, A_2) = 1$
    - ~~$Q(S_0, A_0) \mathrel{+}= \alpha(0.9)^2\delta_2\ ,\ E(S_0, A_0) = (0.9)^3$~~
    - ~~$Q(S_1, A_1) \mathrel{+}= \alpha(0.9)\delta_2\ ,\ E(S_1, A_1) = (0.9)^2$~~
    - $Q(S_2, A_2) \mathrel{+}= \alpha(1)\delta_2\ ,\ E(S_2, A_2) = (0.9)$
  - $t = 3 : (S_3, A_3, R_3, S_4) \rightarrow \delta_3 = R_3 + \gamma Q(S_4, A_4) - Q(S_3, A_3)$
    - $E(S_3, A_3) = 1$
    - ~~$Q(S_0, A_0) \mathrel{+}= \alpha(0.9)^3\delta_3\ ,\ E(S_0, A_0) = (0.9)^4$~~
    - ~~$Q(S_1, A_1) \mathrel{+}= \alpha(0.9)^2\delta_2\ ,\ E(S_1, A_1) = (0.9)^3$~~
    - $Q(S_2, A_2) \mathrel{+}= \alpha(0.9)\delta_3\ ,\ E(S_2, A_2) = (0.9)^2$
    - $Q(S_3, A_3) \mathrel{+}= \alpha(1)\delta_3\ ,\ E(S_3, A_3) = 0.9$

